

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-23T12:30:48.2875899Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260923-123048289.prof
Profil mémoire	memory-20260923-123048289.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	632.77 ms	632765398.2	3870929.8	24

Détail hyperfine

Métrique	Valeur
Moyenne	63.9601 s
Écart-type	1.1980 s
Médiane	64.6781 s
Temps utilisateur	61.9272 s
Temps système	1.8122 s
Min	62.0237 s
Max	64.8095 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	64.18s
Mémoire allouée (pprof)	380.35MB
Mémoire par opération	3870929.8 B/op

Garbage collector Go

Cycles GC	147
Temps GC cumulé	173.590 ms

Plus longue pause	11.000 ms
Temps CPU GC cumulé	657.000 ms
Objectif mémoire maximal	27.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **64.18s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).shouldInfect	41.79s	65.11%	63.39s	98.77%
jeu-de-la-vie/game.(*Board).coordinates (inlin	10.94s	17.05%	10.94s	17.05%
jeu-de-la-vie/game.squaredDistance (inline)	10.62s	16.55%	10.62s	16.55%
jeu-de-la-vie/game.(*Board).Step	0.18s	0.28%	63.77s	99.36%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	63.77s	99.36%
testing.(*B).launch	0	0.00%	63.77s	99.36%
testing.(*B).runN	0	0.00%	63.77s	99.36%

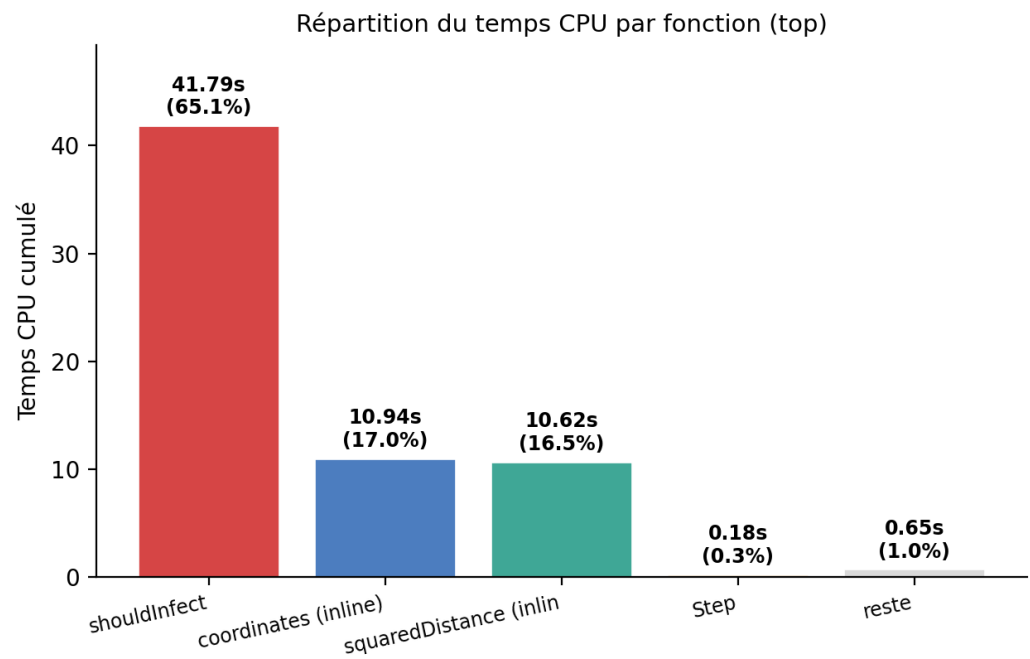


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

File: game.test.exe
 Build ID: C:\Users\sadou\JeuDeLaVie\game.test.exe2026-09-23 14:07:20.0520866 +0200 CEST
 Type: cpu
 Time: 2026-09-23 14:30:48 CEST
 Duration: 65.06s, Total samples = 64.18s (98.65%)
 Showing nodes accounting for 63.53s, 98.99% of 64.18s total
 Dropped 106 nodes (cum <= 0.32s)
 See <https://git.io/HYMW> for how to read the graph

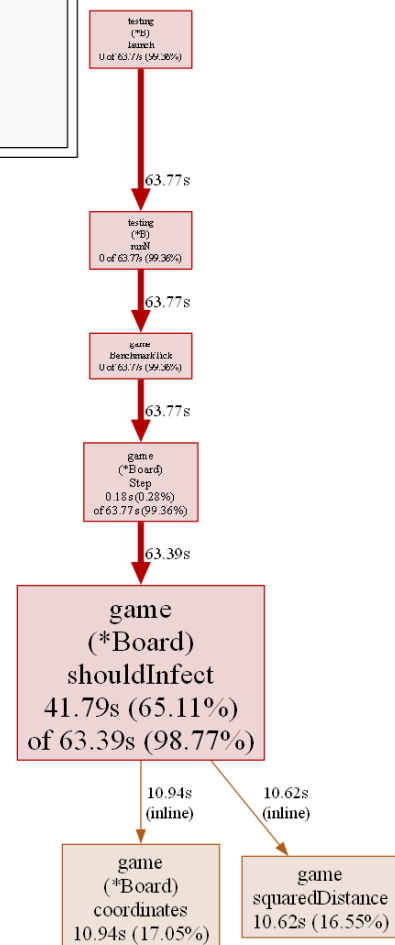


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).shouldInfect**, responsable de **65.1%** du temps CPU total (flat), pour **98.8%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

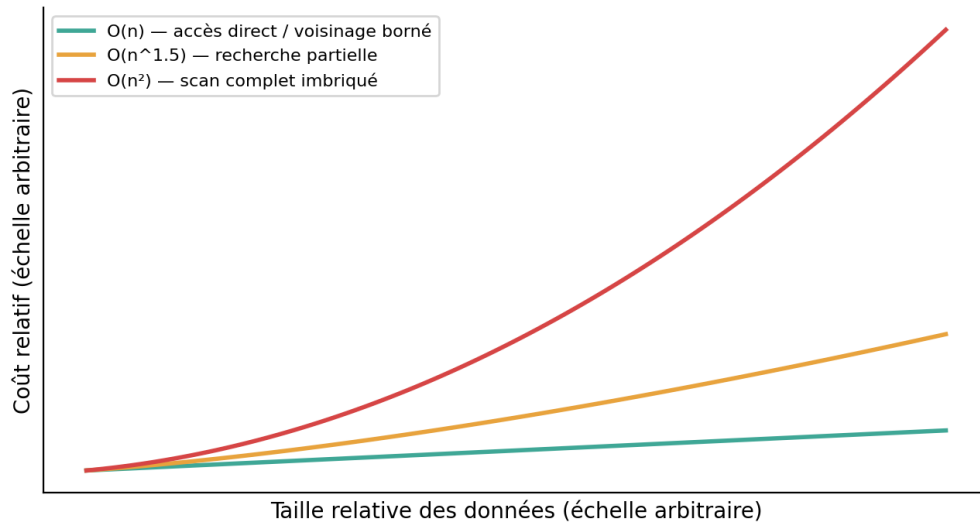


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **380.35MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).Step	367.08MB	96.51%	367.08MB	96.51%
jeu-de-la-vie/game.RandomBoard	5.52MB	1.45%	5.52MB	1.45%
runtime.mallocgc	2MB	0.53%	2MB	0.53%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	373.11MB	98.10%
main.main	0	0.00%	2.22MB	0.58%
runtime.main	0	0.00%	2.72MB	0.72%
runtime/pprof.profileWriter	0	0.00%	2.52MB	0.66%
testing.(*B).launch	0	0.00%	369.15MB	97.06%

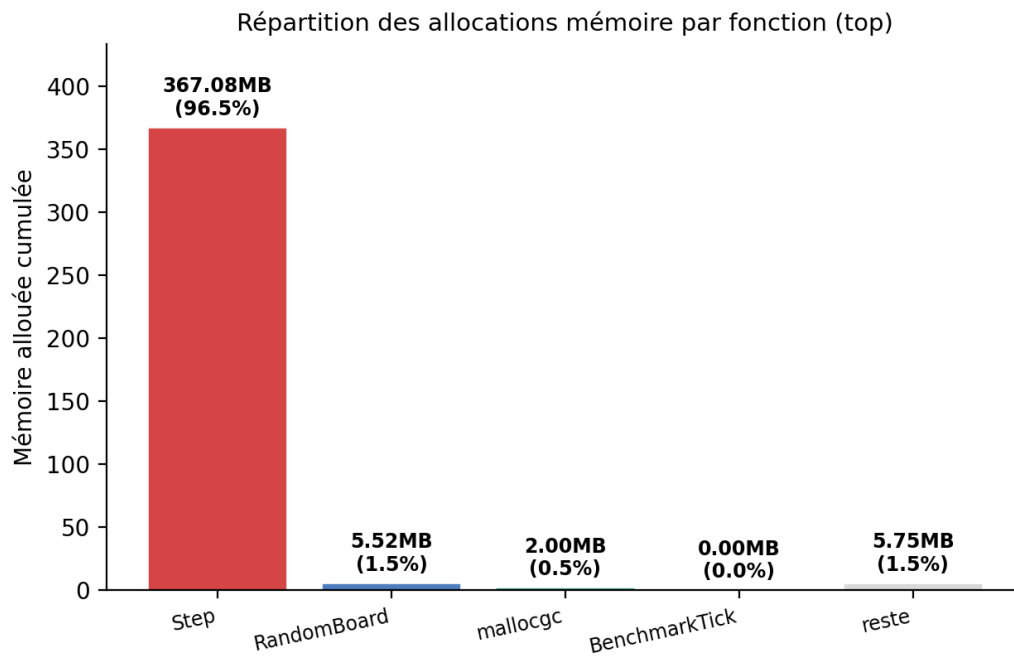


Figure — Répartition des allocations mémoire par fonction.

runtime main 0 of 2.72MB (0.72%)	runtime mallocgc 2MB (U.53%)	pprof profileWriter 0 of 2.52MB (0.66%)
--	------------------------------------	---

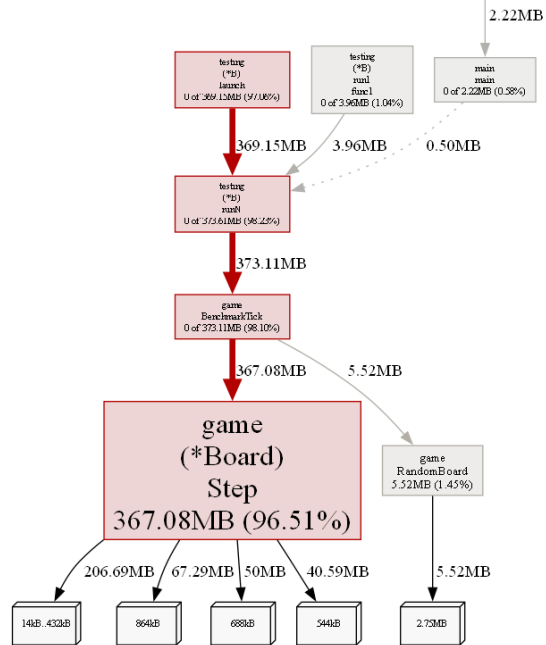


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-23 14:39:25 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.